

Coding Interview Report

Interview Overview

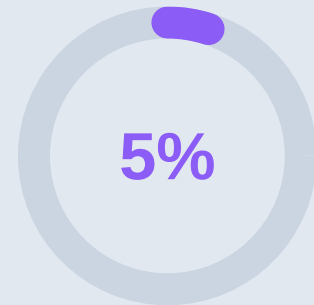
Coding : PYTHON

Started: May 3rd, 2026

Ended: May 3rd, 2026

Duration: 61 minutes

Overall Score



2/3
Qns Attempted

0/3
Qns Passed

Overall Status: **FAILED**

AI Interview Analysis

The candidate demonstrates a foundational understanding of Python and some basic problem-solving skills; however, the results from the coding interview show significant gaps in advanced algorithm design and debugging capabilities. None of the attempted questions were fully resolved, and one question was entirely unattempted. This evidences the need for further preparation in efficient problem-solving techniques and familiarity with algorithmic concepts before being considered for the position.

Strengths

- The candidate displayed clear use of Python syntax in their code submissions.
- A two-pointer approach was attempted in Question 1, which is a common strategy for such problems.
- The use of Python's built-in functions and data structures, such as sets in Question 2, indicates basic proficiency with the language.

Areas for Improvement

- Strengthen problem-solving skills, particularly in implementing algorithms from descriptive problem statements.
- Explore common data structures such as arrays and their manipulation techniques.
- Enhance debugging capabilities to ensure code matches problem requirements.
- Practice foundational algorithms, such as sorting and searching.

Proficient Topics

Code Quality Rating: **3/10**

Coding Problems

1. Reverse Vowels of a String EASY FAILED 1/3 Passed

Given a string `s`, reverse only all the vowels in the string and return it.

The vowels are `'a'`, `'e'`, `'i'`, `'o'`, and `'u'`, and they can appear in both lower and upper cases, more than once.

Example 1:

Input: `s = "IceCreAm"`

Output: `"AceCrelm"`

Explanation: The vowels in `s` are `['i', 'e', 'e', 'A']`. On reversing the vowels, `s` becomes `"AceCrelm"`.

Example 2:

Input: `s = "leetcode"`

Output: `"leotcede"`

Constraints:

- `1 <= s.length <= 3 * 105`
- `s` consist of printable ASCII characters.

Two Pointers

String

Time Complexity

Not applicable

Total Test Cases

3

Space Complexity

Not applicable

Test Cases Passed

1

Total Memory Used

4236

Total Execution Time

0.078

Test Case Results

Status	Input	Output	Expected	Time	Memory
passed	[1],1,[],0	[1]	[1]	0.025	4236
failed	[0],0,[1],1		[1]	0.026	4104
failed	[1,2,3,0,0,0],3,[2,5,6],3		[1,2,2,3,5,6]	0.027	4196

AI Feedback for this Question

The provided code submission appears to address a different problem than the one described in the prompt. The user's code attempts to merge two sorted arrays in-place, which is unrelated to reversing vowels in a string. Consequently, the submission does not fulfill the requirements of the given problem and is non-functional for the specified task.

Code Strengths

- The code demonstrates an understanding of in-place array manipulation.
- The user employs a two-pointer approach effectively for merging arrays.

Areas for Improvement

- The code does not address the problem of reversing vowels in a string as described in the prompt. A new implementation is required.
- There is a typo in the code: ``num1[k]`` should be ``nums1[k]`` and ``num2[j]`` should be ``nums2[j]``.

- Ensure the code passes all test cases by thoroughly testing edge cases and constraints.

Submitted Code (Python (3.8.1))

```
class Solution:
    def merge(self, nums1, m, nums2, n):
        """
        Do not return anything.
        Modify nums1 in-place.
        """
        # Write your code here
        # do not return anything

        i = m - 1
        j = n - 1
        k = m + n - 1

        while i >= 0 and j >= 0:
            if nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                i -= 1
            else:
                nums1[k] = nums2[j]
                j -= 1
                k -= 1

        while j >= 0:
            nums1[k] = nums2[j]
            j -= 1
            k -= 1

        # pass
```

2. Remove Duplicate Letters

MEDIUM

FAILED

1/3 Passed

Given a string `s`, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Example 1:

```
Input: s = "bcabc"
Output: "abc"
```

Example 2:

```
Input: s = "cbacdcbc"
Output: "acdb"
```

Constraints:

- `1 <= s.length <= 104`
- `s` consists of lowercase English letters.

String

Stack

Greedy

Monotonic Stack

Time Complexity

O(N log N)

Total Test Cases

3

Space Complexity

O(N)

Test Cases Passed

1

Total Memory Used

3408

Total Execution Time

0.047

Test Case Results

Status	Input	Output	Expected	Time	Memory
failed	cdadabcc	abcd	adbc	0.017	3408
failed	cbacdcbc	abcd	acdb	0.016	3216
passed	bcabc	abc	abc	0.014	3328

AI Feedback for this Question

The provided code attempts to solve the problem by removing duplicate letters and sorting them lexicographically, but it does not ensure the smallest lexicographical order among all valid results as required by the problem statement.

Code Strengths-

- The code demonstrates an understanding of basic string manipulation techniques.
- The use of `set` to remove duplicates is a valid approach for deduplication.
- The code is concise and syntactically correct.

Areas for Improvement-

- The solution does not account for the order of characters in the original string to ensure the smallest lexicographical order of the result.
- The algorithm needs to implement a stack-based approach to maintain the order while removing duplicates and ensuring lexicographical minimality.
- Consider revising the logic to adhere to the problem constraints and requirements.

Submitted Code (Python (3.8.1))

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        # Write your logic here
        return "".join(sorted(set(s)))
```


3. Sort an Array

MEDIUM

FAILED

0/0 Passed

You are given an integer array `nums`. Your task is to sort the array in **ascending order** and return the sorted array.

The solution must satisfy the following conditions:

- Do **not** use any built-in sorting functions.
- The overall time complexity should be **$O(n \log n)$** .
- Use the **minimum extra space** possible.

Example 1

Input

```
nums = [5,2,3,1]
```

Output

```
[1,2,3,5]
```

Explanation

After sorting, the numbers are arranged in increasing order.

Example 2

Input

```
nums = [5,1,1,2,0,0]
```

Output

```
[0,0,1,1,2,5]
```

Explanation

The array may contain duplicate values, and they should also appear in sorted order.

Constraints

```
1 <= nums.length <= 5 * 10^4  
-5 * 10^4 <= nums[i] <= 5 * 10^4
```

- Array
- Divide and Conquer
- Sorting
- Heap (Priority Queue)
- Merge Sort
- Bucket Sort
- Radix Sort
- Counting Sort

Time Complexity

N/A

Total Test Cases

3

Space Complexity

N/A

Test Cases Passed

0

Total Memory Used

0

Total Execution Time

0

Test Case Results

Status	Input	Output	Expected	Time	Memory
--------	-------	--------	----------	------	--------

Submitted Code (Python (3.8.1))

```
from typing import List

class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        """
        :param nums: List of integers
        :return: Sorted list in O(n log n)
        """
        # Write your logic here (Merge Sort or Heap Sort)
        return nums
```